

Project 4

By Prof. Jie Zhang

Due: 11:59:59 pm, January 21, 2024

In this project, you will explore solid-state drives (SSDs). Specifically, you will learn about the various SSD internal mechanisms (e.g., the flash translation layer (FTL)) through an SSD simulator. Further, you will implement a hybrid-level address mapping scheme in the SSD simulator.

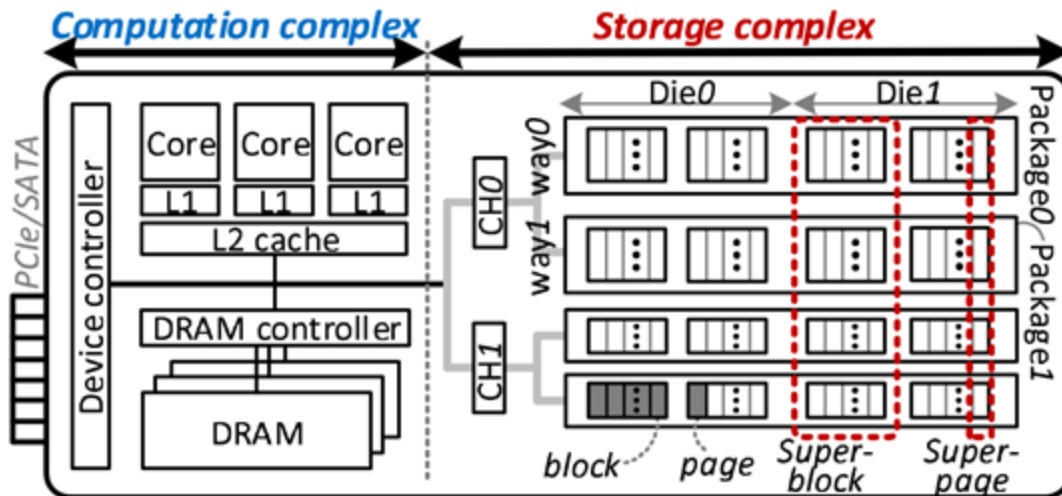
Whenever you face trouble or problems with this project, please contact TAs by WeChat or e-mail. All the following information has been verified on Ubuntu 20.04.1 with Linux Kernel 5.15.0 and Ubuntu 22.04.2 LTS with Linux Kernel 5.19.0.

Note

This project is ***optional***. You are not required to finish this project. If you finish this project, you can get up to 5 bonus points.

Introduction to SSD

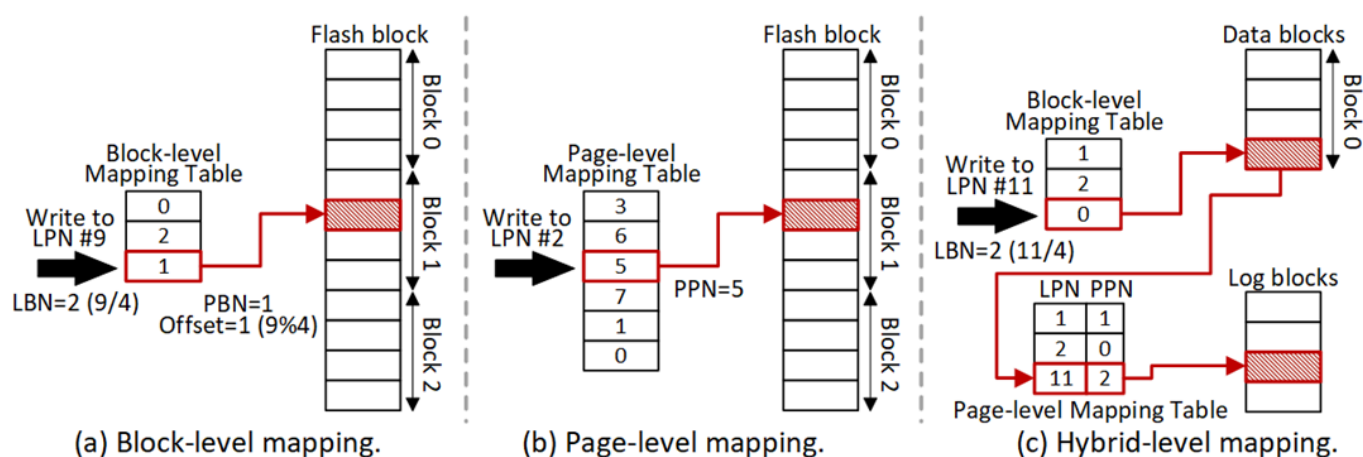
SSDs have become major storage media in modern computing systems. The figure below illustrates the SSD internal architecture, which consists of a computation complex and a storage complex.



The computation complex consists of embedded ARM cores, internal DRAM, and device controllers. Specifically, multiple cores are allocated to flash firmware that controls all I/O services and address translations within an SSD. In addition, the device controller fetches requests from the host and sends responses back via a high-performance communication protocol (e.g., NVMe). SSDs employ embedded DRAM, which stores the address mapping table and I/O queues. To leverage DRAM performance, SSD also buffers the data of requests in DRAM, which hides the long latency imposed by the underlying storage media, namely the **flash**.

The storage complex contains multiple flash **packages**, each containing multiple **dies**, which are connected to the interconnection buses, referred to as **channel**. A set of flash packages across different channels can simultaneously operate. To leverage such parallelism, the flash firmware spreads a host request over multiple dies that have the same offset address but exist across different channels. Each set of flash dies (or packages) is called a **way**. A flash die comprises hundreds to thousands of **blocks**, each containing hundreds of **pages**. A group of multiple physical pages and blocks that span over all internal channels or ways is referred to as **superpage or superblock**, respectively. This organization poses a high SSD internal parallelism, thus achieving superb I/O throughput.

The flash supports read and write operations at **page granularity**. However, due to the physical characteristics, the whole flash block must be erased before writing any page in that block, making flash unsupportable for in-place updates. To address the limitation, the SSD constructs an indirection layer called the **flash translation layer (FTL)**, which maintains a mapping table to record the mapping between the host logical address (LPN) and its corresponding flash physical address (PPN). For an overwrite, the data is written to a pre-erased free block. Subsequently, FTL updates the mapping of the new flash page and invalidates the stale flash page. Depending on the mapping granularity, the FTL mapping schemes can be divided into three categories: block-level, page-level, and hybrid-level mapping, as illustrated in the following figure.



- **Block-level mapping:** In block-level mapping, the LPN is divided into a logical block number (LBN) and a page offset. A block-level mapping table uses the LBN to find the physical block (i.e., PBN) that includes the requested page, while the page offset is used to locate the page in the corresponding block. Due to such a fixed mapping, the page offsets of the logical and physical blocks should be identical, therefore, every overwrite to the same logical page incurs a block-level copy operation. That is, all the data in the corresponding block as well as the new data have to be written into another empty block, resulting in poor overwrite performance.
- **Page-level mapping:** The page-level scheme maps the LPN directly to any PPN by maintaining a logical-physical mapping table (i.e., page-level mapping table in Figure (b)). Although this scheme improves performance with flexible address translation, the mapping table occupies huge amounts of DRAM.
- **Hybrid-level mapping:** To achieve a balance between performance and DRAM consumption, several hybrid-level mapping schemes are proposed. Most of them use a

log block mechanism to store updates. Specifically, flash blocks are divided into data blocks and log blocks. Data blocks represent ordinary storage space, while log blocks are used for storing overwrites. For translating addresses, these schemes maintain a block mapping table for the data blocks and a page mapping table for the log blocks, which consume less DRAM than page-level mapping. When an overwrite request arrives, the new data of the request is written to the log block instead of being stored in the original location, thus avoiding the heavy block copy operation in the block-level mapping. The mapping scheme also determines the specific association between data blocks and log blocks, such as 1-to-1 or N-to-N.

In page-level mapping scheme, when overwrites exhaust all free blocks, **garbage collection** (GC) is triggered to reclaim the used blocks where the SSD selects victim blocks, moves all valid pages to a new free block, and then erases it. On the other hand, in hybrid-level mapping scheme, GC is replaced by the operations of **merging** data blocks and log blocks, as well as recycling log blocks. More specifically, when log blocks are exhausted, the SSD reclaims these blocks by merging them with the associated data blocks. During such merge process, pages are migrated into new blocks and multiple blocks are erased, thereby incurring latency spike.

Introduction to a Precise SSD Simulator

” Cite

Gouk, D., Kwon, M., Zhang, J., Koh, S., Choi, W., Kim, N. S., ... & Jung, M. (2018, October). Amber: Enabling precise full-system simulation with detailed modeling of all SSD resources. In 2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO) (pp. 469-481). IEEE.

[get the pdf](#)

In this part, we will learn about a precise SSD simulator called Amber/SimpleSSD, which is described by the paper cited above. SimpleSSD models embedded ARM cores, DRAMs, and various flash technologies within an SSD. SimpleSSD also includes the full firmware stack of an SSD, including DRAM cache logic, flash firmware (e.g., FTL and Host Interface Layer (HIL)), and diverse standard protocols (e.g., NVMe). To meet various simulation demands, SimpleSSD supports both standalone (without the host system) and full-system (with the host system) simulation environments. Note that in this project, we will use the **standalone** version of SimpleSSD (i.e., SimpleSSD-Standalone) to simulate an SSD. The following content shows the **code structure** of SimpleSSD, where the code in each subfolder simulates the corresponding component. In this project, we mainly focus on **pal** and **ftl** components.

PAL module in SimpleSSD

The PAL in SimpleSSD models the **NAND flash**. The following functions are provided by PAL.

```
class PAL : public StatObject {
private:
    Parameter param;
    AbstractPAL *pPAL;

    ConfigReader &conf;

public:
    PAL(ConfigReader &);
```

```

~PAL();

void read(Request &, uint64_t &);
void write(Request &, uint64_t &);
void erase(Request &, uint64_t &);
};

```

PAL implements `read`, `write`, and `erase` functions that receive the requests from the FTL module and return the latency of the corresponding flash operations. Note that you do **not** need to pay much attention to the details of the PAL. You can view the PAL as a **black-box model**, that is, you provide reasonable parameters for these PAL functions and **get the latency** simulated by the black-box model.

FTL module in SimpleSSD

We then introduce the FTL implementation in SimpleSSD. As shown in the below codes, `simplessd-standalone/simplessd/ftl/abstract_ftl.h` defines the FTL abstract class (i.e., `AbstractFTL`), which contains several virtual functions that **need to be implemented** by the FTL mapping scheme. The `initialize` function sets the **necessary parameters** of the mapping scheme (e.g., the threshold for triggering GC) and **warms up** the SSD by writing a certain amount of data. The `read` and `write` functions describe how the firmware handles read and write requests from the upper layer based on the FTL mapping scheme, respectively. The `getStatus` function returns some **status metrics** for the FTL. Note that your hybrid-level mapping scheme should **inherit AbstractFTL** class and implement all virtual functions required by the abstract class. For convenience, you **do not need to implement specific logic for `trim`, `getStatus`, and `format` functions.**

```

class AbstractFTL : public StatObject {
protected:
    Parameter &param;
    PAL::PAL *pPAL;
    DRAM::AbstractDRAM *pDRAM;
    Status status;

public:
    AbstractFTL(Parameter &p, PAL::PAL *l, DRAM::AbstractDRAM *d)
        : param(p), pPAL(l), pDRAM(d) {}
    virtual ~AbstractFTL() {}
};

```

```

virtual bool initialize() = 0;

virtual void read(Request &, uint64_t &) = 0;
virtual void write(Request &, uint64_t &) = 0;
virtual void trim(Request &, uint64_t &) = 0;

virtual void format(LPNRange &, uint64_t &) = 0;

virtual Status *getStatus(uint64_t, uint64_t) = 0;
};

```

SimpleSSD provides a default **superpage-level FTL mapping scheme**, which can be found in `simplessd-standalone/simplessd/ftl/page_mapping.hh`. `PageMapping` inherits `AbstractFTL` and adds some necessary member variables. Specifically, the `pPAL` is a **pointer to the underlying NAND flash model** (i.e., parallelism abstraction layer (PAL)), which can be used to **obtain the latency** of flash operations from a precise flash model. `table` implements the **superpage-level mapping** from LPNs to PPNs which are described by a flash superpage number and a page offset within the superpage. Note that the superpage-level is slightly different from pure page-level mapping. `blocks` stores flash block status information, such as the page valid information, erase count, and write pointer of blocks. `freeBlocks` is a collection of free blocks, while `nFreeBlocks` stores the number of free blocks, which is used to check whether GC is needed.

```

class PageMapping : public AbstractFTL {
private:
    PAL::PAL *pPAL;

    ConfigReader &conf;

    std::unordered_map<uint64_t, std::vector<std::pair<uint32_t, uint32_t>>>
        table;
    std::unordered_map<uint32_t, Block> blocks;
    std::list<Block> freeBlocks;
    uint32_t nFreeBlocks; // For some libraries which std::list::size() is O(n)
    std::vector<uint32_t> lastFreeBlock;
    Bitset lastFreeBlockIOMap;
    uint32_t lastFreeBlockIndex;

    bool bReclaimMore;
    bool bRandomTweak;
    uint32_t bitsetSize;

```

```

struct {
    uint64_t gcCount;
    uint64_t reclaimedBlocks;
    uint64_t validSuperPageCopies;
    uint64_t validPageCopies;
} stat;
...

```

The rest functions in `PageMapping` are as follows. `PageMapping` simply implements the public interface required by `AbstractFTL` (e.g., `read` and `write`) while placing the detailed logic into the internal functions (e.g., `readInternal` and `writeInternal`). To perform GC, `PageMapping` calculates the `weight` of each block based on `wear-leveling`, selects the victim blocks, and executes the GC operation through `calculateVictimWeight`, `selectVictimBlock`, and `doGarbageCollection` function, respectively.

```

...
float freeBlockRatio();
uint32_t convertBlockIdx(uint32_t);
uint32_t getFreeBlock(uint32_t);
uint32_t getLastFreeBlock(Bitset &);
void calculateVictimWeight(std::vector<std::pair<uint32_t, float>> &,
                          const EVICT_POLICY, uint64_t);
void selectVictimBlock(std::vector<uint32_t> &, uint64_t &);
void doGarbageCollection(std::vector<uint32_t> &, uint64_t &);

float calculateWearLeveling();
void calculateTotalPages(uint64_t &, uint64_t &);

void readInternal(Request &, uint64_t &);
void writeInternal(Request &, uint64_t &, bool = true);
void trimInternal(Request &, uint64_t &);
void eraseInternal(PAL::Request &, uint64_t &);

public:
    PageMapping(ConfigReader &, Parameter &, PAL::PAL *, DRAM::AbstractDRAM *);
    ~PageMapping();

    bool initialize() override;

    void read(Request &, uint64_t &) override;
    void write(Request &, uint64_t &) override;
    void trim(Request &, uint64_t &) override;

    void format(LPNRange &, uint64_t &) override;

```



```

Status *getStatus(uint64_t, uint64_t) override;

void getStatList(std::vector<Stats> &, std::string) override;
void getStatValues(std::vector<double> &) override;
void resetStatValues() override;
};

```

Read operation in PageMapping FTL model

We will use the read operation as an example to introduce how **PageMapping** operates and how it interacts with the upper and lower layers in SimpleSSD. As shown in the following codes in `page_mapping.cc`, **PageMapping** first looks up the mapping table (i.e., `table`) to translate the LPN to the PPN which is stored in `mappingList`. It then performs the latency simulation of DRAM access via the `pDRAM` member.

```

void PageMapping::readInternal(Request &req, uint64_t &tick) {
    PAL::Request palRequest(req); // produce pal request
    uint64_t beginAt;
    uint64_t finishedAt = tick;

    auto mappingList = table.find(req.lpn); // find the mapping list

    if (mappingList != table.end()) {
        if (bRandomTweak) {
            pDRAM->read(&(*mappingList), 8 * req.ioFlag.count(), tick);
        }
        else {
            pDRAM->read(&(*mappingList), 8, tick); // simulate DRAM access latency
        }
        ...
    }
}

```

PageMapping then locates the data required by **the upper layer request** in the superpage and subsequently **constructs the underlying layer request** (i.e., `palRequest`) based on the address of the requested data. Further, **PageMapping** sends the `palRequest` to the NAND flash model via the `pPAL` pointer and obtains the flash read latency in the `beginAt` variable.

```

...
for (uint32_t idx = 0; idx < bitsetSize; idx++) {

```

```

        if (req.ioFlag.test(idx) || !bRandomTweak) {
            auto &mapping = mappingList->second.at(idx); // locate the exact PPN in
the mapping list

            if (mapping.first < param.totalPhysicalBlocks &&
                mapping.second < param.pagesInBlock) {
                palRequest.blockIndex = mapping.first; // constructed by PAL::Request
palRequest(req);
                palRequest.pageIndex = mapping.second;

                if (bRandomTweak) {
                    palRequest.ioFlag.reset();
                    palRequest.ioFlag.set(idx);
                }
                else {
                    palRequest.ioFlag.set();
                }

                auto block = blocks.find(palRequest.blockIndex);

                if (block == blocks.end()) {
                    panic("Block is not in use");
                }

                beginAt = tick;

                block->second.read(palRequest.pageIndex, idx, beginAt); // update
block metadata
                pPAL->read(palRequest, beginAt); // after this, beginAt stores the
finish time of the PAL request

                finishedAt = MAX(finishedAt, beginAt);
            }
        }
    }
    ...

```

PageMapping updates the request latency by taking the maximum value of the returned read latency. **PageMapping** also adds the firmware latency to the request latency. But in this project, **you can neglect the firmware latency**. Finally, the request latency is returned to the upper layer.

```

...
tick = finishedAt; // update the tick argument to pass the finish time

```

```

back, do not need return
    tick += applyLatency(CPU::FTL__PAGE_MAPPING, CPU::READ_INTERNAL);
}
}

```

Info

You may notice that the code checks the **bRandomTweak** for several times and does different behaviors based on its value. In general, random-tweak **enables page-level reads and writes under superpage-level mapping**. The **idx**, **ioFlag** and **bitsetSize** are all used for this purpose. The **table** also saves vectors of block-page pair for each mapping. In this project, we **turn this function off**, and you **do not need to implement any features for this function**.

Write operation in PageMapping FTL model

Then we introduce the write operations performed by **PageMapping**. Specifically, **PageMapping** employs **Block** class to manage the flash blocks currently in use within the SSD. As we describe before, this class encompasses various information, including the validity status of individual pages and LPN associated with each page. Moreover, **Block** **offers multiple functions** that **emulate real flash operations**, such as writing a page to a flash block (**write** function) and invalidating a page within the flash block (**invalidate** function). It is crucial to **carefully examine its code as FAST frequently queries the flash block information**. When **PageMapping** starts processing a write request, it first invalidates the obsolete page. This process involves finding the previous mapping from table, subsequently locating the corresponding block, and finally invoking the **invalidate** function.

```

void PageMapping::writeInternal(Request &req, uint64_t &tick, bool sendToPAL) {
    PAL::Request palRequest(req);
    std::unordered_map<uint32_t, Block>::iterator block;
    auto mappingList = table.find(req.lpn);
    uint64_t beginAt;
    uint64_t finishedAt = tick;
    bool readBeforeWrite = false;

```

```

if (mappingList != table.end()) { // exists old page on this LPN
    for (uint32_t idx = 0; idx < bitsetSize; idx++) {
        if (req.ioFlag.test(idx) || !bRandomTweak) {
            auto &mapping = mappingList->second.at(idx);

            if (mapping.first < param.totalPhysicalBlocks &&
                mapping.second < param.pagesInBlock) {
                block = blocks.find(mapping.first);

                // Invalidate current page
                block->second.invalidate(mapping.second, idx);
            }
        }
    }
} else { // no old page on this LPN
    // Create empty mapping
    auto ret = table.emplace(
        req.lpn,
        std::vector<std::pair<uint32_t, uint32_t>>(
            bitsetSize, {param.totalPhysicalBlocks, param.pagesInBlock}));

    if (!ret.second) {
        panic("Failed to insert new mapping");
    }

    mappingList = ret.first;
}
...

```

It proceeds by allocating a free flash page for the upcoming write via `getLastFreeBlock` function. Notice that to optimize performance, SimpleSSD customizes the allocation policy to suit the flash parallelism. But you can [simplify this allocation process](#) in your implementation. This can be achieved by maintaining [a linked list of free blocks](#), where each write utilizes the block at the head, and any reclaimed free block is inserted at the end (i.e., the [FIFO](#) policy).

```

...
block = blocks.find(getLastFreeBlock(req.ioFlag));

if (block == blocks.end()) {
    panic("No such block");
}

if (sendToPAL) {

```

```

    if (bRandomTweak) {
        pDRAM->read(&(*mappingList), 8 * req.ioFlag.count(), tick);
        pDRAM->write(&(*mappingList), 8 * req.ioFlag.count(), tick);
    }
    else {
        pDRAM->read(&(*mappingList), 8, tick); // simulate DRAM access latency.
        read-modify-write the mapping table.
        pDRAM->write(&(*mappingList), 8, tick);
    }
}

```

Finally, similar to a read operation, the **PageMapping** emulates a write operation in the block by validating the flash page and recording the corresponding LPN via write function. Subsequently, a **palRequest** is dispatched to the underlying PAL to acquire the write latency. It is worth noting that **sendToPAL** is utilized to distinguish whether the write request originates from the warmup phase or the emulation phase. The warmup phase is specifically designed to replicate the real SSD state after a prolonged period of operation, and it constructs write requests to change the SSD internal states (e.g., FTL mappings). To rapidly reach the steady state, write requests in the warmup phase does not need to be forwarded to the PAL, instead, they solely necessitate modification to metadata within the FTL. Similarly, in your implementation, it is imperative to support the warmup operation to emulate internal states in real SSDs.

```

...
for (uint32_t idx = 0; idx < bitsetSize; idx++) {
    if (req.ioFlag.test(idx) || !bRandomTweak) {
        uint32_t pageIndex = block->second.getNextWritePageIndex(idx); // find an
empty page to write
        auto &mapping = mappingList->second.at(idx);

        beginAt = tick;
        block->second.write(pageIndex, req.lpn, idx, beginAt);

        // Read old data if needed (Only executed when bRandomTweak = false)
        // Maybe some other init procedures want to perform 'partial-write'
        // So check sendToPAL variable
        if (readBeforeWrite && sendToPAL) {
            palRequest.blockIndex = mapping.first;
            palRequest.pageIndex = mapping.second;

            // We don't need to read old data
            palRequest.ioFlag = req.ioFlag;

```

```

        palRequest.ioFlag.flip();
        pPAL->read(palRequest, beginAt);
    }

    // update mapping to table
    mapping.first = block->first;
    mapping.second = pageIndex;

    if (sendToPAL) {
        palRequest.blockIndex = block->first;
        palRequest.pageIndex = pageIndex;

        if (bRandomTweak) {
            palRequest.ioFlag.reset();
            palRequest.ioFlag.set(idx);
        }
        else {
            palRequest.ioFlag.set();
        }
        pPAL->write(palRequest, beginAt);
    }
    finishedAt = MAX(finishedAt, beginAt);
}
}

// Exclude CPU operation when initializing
if (sendToPAL) {
    tick = finishedAt;
    tick += applyLatency(CPU::FTL__PAGE_MAPPING, CPU::WRITE_INTERNAL);
}

```

Garbage collection in PageMapping FTL model

After the write operation finishes, the amount of free blocks may drop below a threshold. Garbage collection is a process to find some victim blocks, move the valid pages from these blocks into free blocks, and erase these blocks, making them free blocks. This process is performed in `writeInternal`:

```

// GC if needed
// I assumed that init procedure never invokes GC
static float gcThreshold = conf.readFloat(CONFIG_FTL,

```

```

FTL_GC_THRESHOLD_RATIO);

if (freeBlockRatio() < gcThreshold) {
    if (!sendToPAL) {
        panic("ftl: GC triggered while in initialization");
    }

    std::vector<uint32_t> list;
    uint64_t beginAt = tick;

    selectVictimBlock(list, beginAt);

    debugprint(LOG_FTL_PAGE_MAPPING,
               "GC    | On-demand | %u blocks will be reclaimed", list.size());

    doGarbageCollection(list, beginAt);

    debugprint(LOG_FTL_PAGE_MAPPING,
               "GC    | Done | %" PRIu64 " - %" PRIu64 " (%" PRIu64 ")", tick,
               beginAt, beginAt - tick);

    stat.gcCount++;
    stat.reclaimedBlocks += list.size();
}

```

The `selectVictimBlock` selects suitable blocks for being victim and push them into the `list`. The `doGarbageCollection` performs actual PAL requests. It firstly constructs three lists, recording the read, write and erase requests that are going to be performed, respectively.

```

void PageMapping::doGarbageCollection(std::vector<uint32_t> &blocksToReclaim,
                                       uint64_t &tick) {
    PAL::Request req(param.ioUnitInPage);
    std::vector<PAL::Request> readRequests;
    std::vector<PAL::Request> writeRequests;
    std::vector<PAL::Request> eraseRequests;
    std::vector<uint64_t> lpns;
    Bitset bit(param.ioUnitInPage);
    uint64_t beginAt;
    uint64_t readFinishedAt = tick;
    uint64_t writeFinishedAt = tick;
    uint64_t eraseFinishedAt = tick;
    ...
}

```

Then, it builds the requests for each block in the list (i.e., `blocksToReclaim`). For each valid page in the block, it reads the page out, writes it to a free page, and updates the corresponding metadata. Then it erases all the victim blocks.

```
// For all blocks to reclaim, collecting request structure only
for (auto &iter : blocksToReclaim) {
    auto block = blocks.find(iter);

    if (block == blocks.end()) {
        panic("Invalid block");
    }

    // Copy valid pages to free block
    for (uint32_t pageIndex = 0; pageIndex < param.pagesInBlock; pageIndex++) {
        // Valid?
        if (block->second.getPageInfo(pageIndex, lpns, bit)) {
            if (!bRandomTweak) {
                bit.set();
            }

            // Retrive free block
            auto freeBlock = blocks.find(getLastFreeBlock(bit));

            // Issue Read
            req.blockIndex = block->first;
            req.pageIndex = pageIndex;
            req.ioFlag = bit;

            readRequests.push_back(req);

            // Update mapping table
            uint32_t newBlockIdx = freeBlock->first;

            for (uint32_t idx = 0; idx < bitsetSize; idx++) {
                if (bit.test(idx)) {
                    // Invalidate
                    block->second.invalidate(pageIndex, idx);

                    auto mappingList = table.find(lpns.at(idx));

                    if (mappingList == table.end()) {
                        panic("Invalid mapping table entry");
                    }

                    pDRAM->read(&(*mappingList), 8 * param.ioUnitInPage, tick);
                }
            }
        }
    }
}
```



```

        auto &mapping = mappingList->second.at(idx);

        uint32_t newPageIdx = freeBlock->second.getNextWritePageIndex(idx);

        mapping.first = newBlockIdx;
        mapping.second = newPageIdx;

        freeBlock->second.write(newPageIdx, lpns.at(idx), idx, beginAt);

        // Issue Write
        req.blockIndex = newBlockIdx;
        req.pageIndex = newPageIdx;

        if (bRandomTweak) {
            req.ioFlag.reset();
            req.ioFlag.set(idx);
        }
        else {
            req.ioFlag.set();
        }

        writeRequests.push_back(req);

        stat.validPageCopies++;
    }
}

stat.validSuperPageCopies++;
}

// Erase block
req.blockIndex = block->first;
req.pageIndex = 0;
req.ioFlag.set();

eraseRequests.push_back(req);
}

```

After this process, all the requests are actually performed.

```

// Do actual I/O here
// This handles PAL2 limitation (SIGSEGV, infinite loop, or so-on)
for (auto &iter : readRequests) {
    beginAt = tick;

```

```
pPAL->read(iter, beginAt);

readFinishedAt = MAX(readFinishedAt, beginAt);
}

for (auto &iter : writeRequests) {
    beginAt = readFinishedAt;

    pPAL->write(iter, beginAt);

    writeFinishedAt = MAX(writeFinishedAt, beginAt);
}

for (auto &iter : eraseRequests) {
    beginAt = readFinishedAt;

    eraseInternal(iter, beginAt);

    eraseFinishedAt = MAX(eraseFinishedAt, beginAt);
}

tick = MAX(writeFinishedAt, eraseFinishedAt);
tick += applyLatency(CPU::FTL__PAGE_MAPPING, CPU::DO_GARBAGE_COLLECTION);
}
```

SimpleSSD Setup and Configuration

The [SimpleSSD document](#) shows detailed procedures to compile and run SimpleSSD. The first thing you need to do is download SimpleSSD. For your convenience, we've provided a code skeleton for you, so you do not need to download the codes from Github. You can download the package from [PKU Disk](#).

To build SimpleSSD, you will need CMake. You can install it via the following commands:

```
sudo apt-get update
sudo apt-get install cmake --fix-missing
```

Next, you can setup the workspace and build the program **simpl SSD-standalone** :

```
cd simplessd-standalone
cmake -DDEBUG_BUILD=off .
make -j 8
```

After successfully building SimpleSSD, you can use the following command to run SimpleSSD. SimpleSSD runs the workload according to the settings in the Simulation configuration file and SimpleSSD configuration file. It outputs the simulation results and logs to the path specified by the Output directory.

```
# Usage: simplessd-standalone <Simulation configuration file> <SimpleSSD
configuration file> <Output directory>
./simplessd-standalone ./config/sample.cfg ./simplessd/config/sample.cfg
./output
```

We've provide the default simulation configuration file and SimpleSSD configuration file, which are located in `simplessd-standalone/config/sample.cfg` and `simplessd-standalone/simplessd/config/sample.cfg`, respectively.

The **simulation configuration file** sets the key parameters of the simulation environment. Specifically, `LogFile`, `DebugLogFile`, and `LatencyLogFile` set the output paths of the statistic log, debug log, and latency log, respectively. These log files help you observe the behavior of the SSD. For example, `DebugLogFile` records the log output during simulation, which can track the processing path of each request, while `LatencyLogFile` holds statistics for each request in the format of `(request ID, address offset, request length, latency (in ps))`. Additionally, SimpleSSD provides an I/O generator and a trace replayer to generate workloads for evaluations, and which one to use is decided by `Mode` in this file. You can use them to evaluate the performance of your FTL scheme implementation. Note that the `log file and debug log file` can be very `large`. You can empty the parameter fields to turn off logging.

```
## Select mode
# Possible values:
# 0: Request generator mode (all configs in [trace] will be ignored)
# 1: Trace replayer mode (all configs in [generator] will be ignored)
Mode = 1

## Statistic log period
# Print statistic log periodically
# 0 means no log printout
```

```
# Unit: ms (millisecond) in simulation time (not real time)
LogPeriod = 10

## Log file
# Set output path of statistic log and debug log
# <empty value> means no log printout
# Possible keywords:
#   STDOUT: Standard Output
#   STDERR: Standard Error
LogFile = log.txt
DebugLogFile = debug.txt

## I/O Log file
# Set output path of latency log
# <empty value> means no log printout
LatencyLogFile = latency.csv
```

we provide 4 traces obtained from the FIU traces, plus a simple traces for you to debug. These workload exhibits complicated I/O patterns that can check the correctness of your implementation. In addition, workload labels each request as sequential or random. Based on this information and combined with latency data, the correctness of your implementation can be checked. For example, sequential writes should benefit from the SW log block design. You can use these traces by setting **Mode = 1** and modify the **File** parameter:

```
[trace]
## Trace file
File = ./traces/simple
```

You will also need to run microbenchmark via the request generator mode. This mode will automatically generate requests according to the configuration. The **io_size** defines the total size of all requests. **readwrite** defines the pattern of the requests. Sequential read/write, random read/write and mixed sequential/random I/O are possible configurations. **rwmixread** defines the ratio of read requests in all requests when running under mixed patterns. **blocksize** defines the size of each individual request.

```
[generator]
## This request generator will work similar to Flexible I/O tester
# Definition of Types:
#   str:   String, a sequence of alphanumeric characters
#   int:   SI integer, a sequence of numeric characters
#   Possible suffix (case sensitive):
#       10^3 base: k, m, g, t
```

```

#           2^10 base: K, M, G, T
# float: Floating point number, a sequence of numeric characters
#         with one (or none) of decimal point
# bool: Boolean, a true or false value.
#       Possible value (case insensitive):
#       True: Non-zero number, T, True, Y, Yes
#       False: Otherwise
# time: SI integer, a sequenced of numeric characters
#       If no suffix provided, value will treated as pico-second.
#       Possible suffix (case sensitive):
#       s, ms, us, ns, ps

## Total size to I/O = int
# Only valid when time_based = false
io_size = 128M # 16 * 2^20 bytes

## Type of I/O = str
# Possible values:
# read: Sequential read
# write: Sequential write
# randread: Random read
# randwrite: Random write
# readwrite: Mixed sequential I/O
# randrw: Mixed random I/O
readwrite = write

## Percentage of mixed I/O = float
# If readwrite is not readwrite nor randrw, this value will ignored
# 0 <= value <= 1
rwmixread = 0.5 # 50% of read I/O

## Block size = int
blocksize = 4K

```

Other parameters in the simulation configuration file can be left unchanged during this project.

The **SimpleSSD configuration file** sets the key parameters of the simulated SSD, including the embedded cores, NVMe, FTL, ICL, and PAL. This file provides a detailed explanation for each parameter, and you can refer to this file to learn how to set the parameters. Nevertheless, the only necessary parameter in this project is **MappingMode**. You can leave others unchanged.

```

[ftl]
## Set mapping method

```

```
# Possible values:  
# 0: Page level mapping  
# 1: FAST mapping  
MappingMode = 1
```

Task: Read Paper, Implement and Evaluate FAST

” Cite

Lee, S. W., Park, D. J., Chung, T. S., Lee, D. H., Park, S., & Song, H. J. (2007). A log buffer-based flash translation layer using fully-associative sector translation. ACM Transactions on Embedded Computing Systems (TECS), 6(3), 18-es.

[get the pdf](#)

The task in this project is to read the paper cited above, implement the FAST hybrid-level mapping scheme in SimpleSSD, and evaluate it.

FAST mapping scheme: a brief introduction

The FAST scheme leverages a hybrid-level mapping, which divides the physical blocks into data blocks and log blocks.

The **data blocks** use **block-level mapping**. That is, when it maps a page with LPN ($= \text{LBN} * \text{pagePerBlock} + \text{PageOffset}$), the mapping table records only LBN-PBN mapping pairs. The PageOffset remains unchanged during the mapping.

The **log blocks** use **page-level mapping**. The mapping table records LPN-PPN pairs, and the PageOffset of the LPN and PPN can be different.

Further more, FAST divides log blocks into **Sequential Write Block** and **Random Write Blocks**. The entire system contains **a single SW block**. It is used to **buffer sequential writes**. It also leverages **block-level mapping**, so when it is merged with data blocks, there is possibility to eliminate operations to speed up the merging. **RW blocks uses page-level mapping**, and the total number of RW blocks are constrained (set to **6** in this project).

There are four types of operation in FAST:

1. **Read.** When reading a page, FAST first checks if any **log block** (SW or RW block) contains this page. If so, it reads the page from the block. Otherwise, it finds the **data block** by mapping the LBN to PBN. If there exist a data block that maps to the LBN, and the physical page on Page Offset is valid, FAST reads the page from it.
2. **Write.** When writing a page, FAST first checks if the corresponding physical page, i.e., $PBN * pagePerBlock + PageOffset$, is **free** (hasn't been written since last erase). If so, FAST writes the page to it. If not, then FAST writes it to a log block. Firstly, it checks whether this is a **sequential write** by checking two conditions: (1) PageOffset is 0, no matter what LBN; (2) the LBN is equal to the LBN of **current SW block**, and the PageOffset is exactly the **first free physical page** of the SW block (for example, SW block = [0, 1, free, free], writing page 2). If this is a sequential write, FAST writes the page to SW block. Specially, under condition (1), if the write LBN is **not** the LBN of SW block, the SW block should be **merged first**. If this is not a sequential write, FAST finds a **non-full RW block** and writes the page to the **first free page** of it.
3. **SW block merge.** "Merge" is the operation that transforms a log block and a few corresponding data blocks into new data blocks and erases these old blocks. When a write with $PageOffset == 0$ comes, but the LBN is **not** the current LBN of SW block, FAST merges the SW block first. There are two conditions: (1) the SW block is full of valid pages. Under this condition, the SW block can be "switched" to a data block. FAST finds the old data block of this LBN, modifies the mapping table to let the LBN maps to current SW block, and **erase the old data block** into a new free block. (2) The SW block is not full. Under this condition, FAST finds the **old data block of this LBN**, reads **all valid** pages from the data and SW blocks, **allocates a new free block** as new data block, writes the valid pages to the new data block, and **erases the old data and SW blocks**. After both conditions, FAST allocates a free block as the new SW block.
4. **RW block merge.** When **all RW blocks are full** (with valid and invalid pages, no more free page), FAST choose a victim RW block to be merged. A RW block may contains logical pages that belong to different logical blocks. FAST first finds **all related LBNs of the LPNs in the victim block**. Then, it reads all valid pages that belong to these LBNs from all RW blocks and corresponding data blocks (will scan RW blocks, but check the mapping table to find the data blocks to read). It then allocates multiple free blocks for these LBNs, one for each, and write the valid pages to them. Lastly, it erases the old data blocks and the victim block, then allocates a new free RW block.

Provided contents in code package

We've provided a few contents in the code package for your convenience (you can download the package from [PKU Disk](#)):

1. `config/simulation_sample.cfg`: A sample configuration file.
2. `config/<trace name>.cfg`: Four configuration files that configure the program to run on the traces.
3. `simplessd/config/ssd_sample.cfg`: A sample configuration file.
4. `simplessd/config/<base/fast>.cfg`: Two configuration files that configure the program to run with base mapping (`PageMapping`) and fast mapping.
5. `traces/`: Four traces from FIU and one simple trace with 6 requests.
6. `compile`: A bash script that calls CMake to setup and compile `simplessd`. It will compile in both release mode and debug mode.
7. `test`: A bash script that automatically simulates the four traces with base/fast mapping. You can run this script in background (`./test &`). It may take ~10 minutes to complete.
8. `plot.py`, `plot`: Python and bash scripts that draw the latency plots for each setting, based on latency log output `latency.csv`. You can also open the CSV files in Excel and draw the plot. Note that the other log outputs (`log.txt` and `debug.txt`) can be very large, you can modify the configuration files, empty the corresponding fields to turn the log off.
9. `simplessd/ftl/fast.hh`, `simplessd/ftl/fast.cc`: Code skeleton. You can only work under these two files.

Code skeleton

We've provide a detailed code skeleton for you. Basically, `fast.hh` defines all the classes and interfaces. There are six functions that require you to complete in `fast.cc` (You can search for `TODO: Lab4` to see the exact location):

```
void invalidateOldPage(LBN lbn, POff offset);
std::pair<PBN, POff> findPageInRWBlocks(LPN lpn);
bool findPageInSWBlock(LBN lbn, POff offset);
PBN findPageInDataBlocks(LBN lbn, POff offset);
bool pageIsFreeInDataBlock(LBN lbn, POff offset);
bool canWriteToSWBlockAfterMerge(LBN lbn, POff offset, uint64_t &tick,
```

```
bool sendToPAL);  
std::pair<PBN, POff> findFreePageInRWBlocks(uint64_t &tick, bool sendToPAL);
```

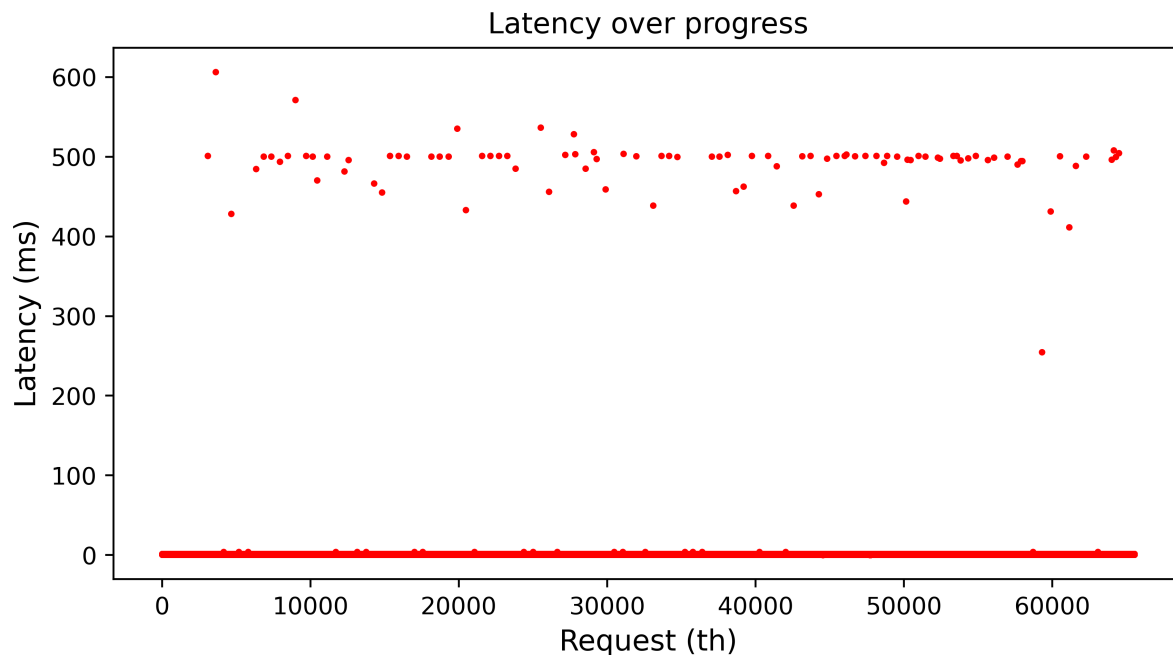
Others are already implemented. You can check the comments for the detailed description. You are not required to simulate the DRAM access latency in this project for simplicity.

Info

The interfaces in SimpleSSD leverage a unique implementation to simulate the latency of each request. Basically, The request is passed from the top to the bottom by a serial of interfaces like **read** and **write**. Each interface receive a reference to a variable, namely **uint64_t &tick**. The interfaces modify this variable directly when there is a new latency component, like **tick += applyLatency(CPU::FTL__PAGE_MAPPING, CPU::READ_INTERNAL);**. They can also pass this variable to other interfaces to gather the latency of next layer, like **pPAL->read(palRequest, tick);**. You can use this method to implement FAST.

Desired outputs

Compared to **PageMapping**, your implementation of FAST should exhibits latency spikes that are **much higher than the average latency**. The latency plots can be like this:



Deliverables

In this project, please include the following contents in your **report**:

- Codes of your implementation.
- The **comparison of your implementation with *PageMapping*** in SimpleSSD. Specifically, show the latency plots for each traces.

Submission

Submission

- Please submit the deliverables for each part in order and clearly defined in a report form (e.g., PDF or Word). Please give a brief description of the results in your report. You also **NEED** to submit your codes and paper summary.
- Please pack your report, codes and summary in an archive and submit it as an attachment at course.pku.edu.cn. The title of the attachment should be Student-ID_Name_Proj4 (e.g., 123456789_WangXiaoming_Proj4).
- You can also send the archive to ca2024fall@163.com. The titles of your email AND attachment should both be Student-ID_Name_Proj4.
- **DO NOT PLAGIARIZE.** We will select 10 students randomly and ask them to answer our questions related to their results.

Late policy

- You will be given **3 slip days** (shared by all projects), which can be used to extend project deadlines, e.g., 1 project extended by 3 days or 3 projects each extended by 1 day.
- Projects are due at 23:59:59, no exceptions; 20% off per day late, 1 second late = 1 hour late = 1 day late.

Deliverables

In this project, please include the following contents in your **report**:

- Codes of your implementation.
- The **comparison of your implementation with *PageMapping*** in SimpleSSD. Specifically, show the latency plots for each traces.

Grading

In this project, the grading is partially based on the performance of your implementation. The details are (5 points in total):

- A FAST implementation that can finish the simulation of the four traces without fatal error:
 - **4pts.**
- **PLUS** resonable performance of your implementation:
 - **5pts.**